

Technische Hochschule Ingolstadt
Fakultät Elektro- und Informationstechnik

Sensor Network Technologies and Sensor Fusion

Features for SLAM
Grid Mapping

Jan 08, 2025

Welcome to the lecture!

Prof. Dr.-Ing. Björn Giesler

Technische Hochschule Ingolstadt

AIMotion, AI and Sensor Data Fusion

Email: bjoern.giesler@thi.de

- **Please register for the lecture on Moodle!**
  - Link: https://moodle.thi.de/course/view.php?id=10580
  - Key: **SensNT+DF#WS25**
- **News on Moodle:**
  - Slides for the lecture (upcoming lectures marked as WIP and are subject to change!)
  - Formulary, Exercises (Solutions provided after discussion in the lecture)
  - Exercises are optional, one task / sheet per week, but are highly recommended.
    They can be handed in for correction.
  - I will *try* to publish exercises before Wednesday‘s lecture so that initial questions can be discussed on Thursday.
- **Exam: Written exam, 90 minutes**
- **Consultation hours: Please arrange via e-mail to bjoern.giesler@thi.de.**
- **Please also don‘t hesitate to send questions, lecture feedback, change requests, etc.!**

- **Previous lectures:**
  - Mathematical details of Particle Filters
  - Joint Probability Data Association Filters
  - JPDAs and Particle Filters
  - Simultaneous Localization and Mapping (based on Kalman Filters)
- **This week:**
  - Simultaneous Localization and Mapping (based on Particle Filters)
  - Overview over features for SLAM
  - Occupancy Grids and Grid-based SLAM
- **Upcoming:**
  - (DNN-based) early fusion approaches.

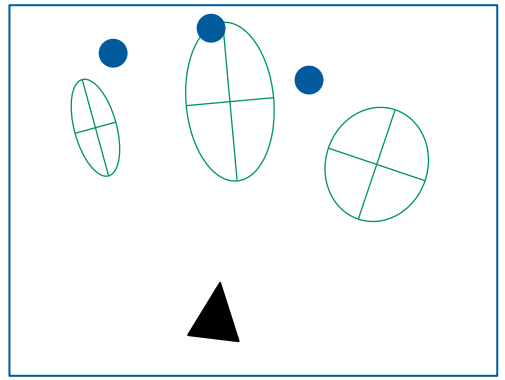
Let's talk exam!

- **I want to know what's in your brain! My priority is always**
  1. have you understood the concept?
  2. ...thoroughly enough so you could implement it practically?
  3. and last, are the numerical results you obtain correct?
- **No formulary, personal notes, tablets, laptops, AI assistants or personal humanoid messenger or telepresence robots allowed.**
- **Pocket calculators are a necessity.**
- **There will likely be some hand calculation with the help of pocket calculators.**
- **No coding of course, but questions about algorithms in pseudo code are possible.**
- **Long answer questions possible. Multiple choice questions possible. Formula questions possible.**
- **Basically everything we discussed in the lecture is relevant. Do not rely on only the exercise sheets or programming exercises to prepare.**
- **I will be fair!**

Landmark Features for SLAM

Landmark Features for SLAM

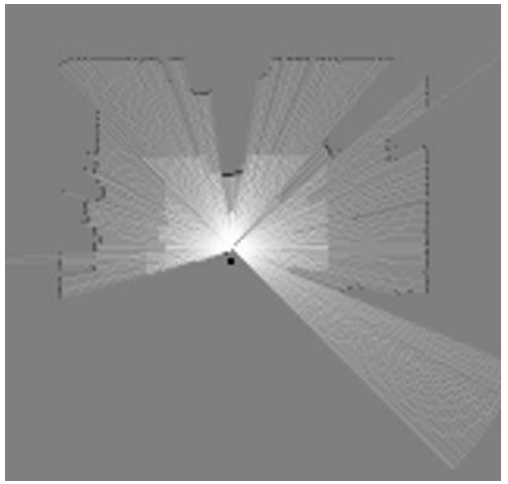
- **We have talked a lot about landmark matches and observations.**
- ***What's a landmark?***
- **Basic definition of a landmark (from Webster's Dictionary:**
  A conspicuous object that marks a locality.
- **Our definition: Any object that, *within our sensor constraints*,**
  - can be easily recognized and associated to a map feature.
  - can be *localized to a single point* as precisely as possible
    (2D point if we are moving in a 2D plane, 3D point if we are moving in 3D)
    ideally better than an individual raw sensor measurement.



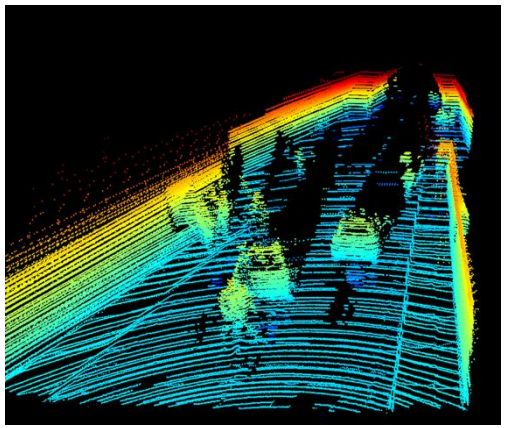
- **Often distinguished between *natural landmarks* and *artificial landmarks:***
  - Artificial landmarks are placed specifically to aid localization.
  - Natural landmarks occur naturally in the environment.
- **Most of this stuff is far beyond the scope of this lecture series, so only a very high level overview is given here.**

Landmark Features for Lidar

- **Lidar is very popular for navigation / SLAM in structured environments due to its precision.**
- **Comes in two basic variants –**
- **2D lidar:**
  - Scans in a single plane, delivering a "cross section" of the environment.
  - Often very high opening angle – up to 360°
  - Typically used in indoor / planar motion environments (robots in buildings).
  - Price has come down tremendously (<100€).
- **3D lidar:**
  - Scans in multiple planes, delivering a full 3D point cloud.
  - Typically used in outdoor / full 3D environments (driverless cars, drones).
  - Opening angle often limited but still large – Luminar: 120°
  - Still rather expensive (>1000€ depending on technology).



2D lidar single scan



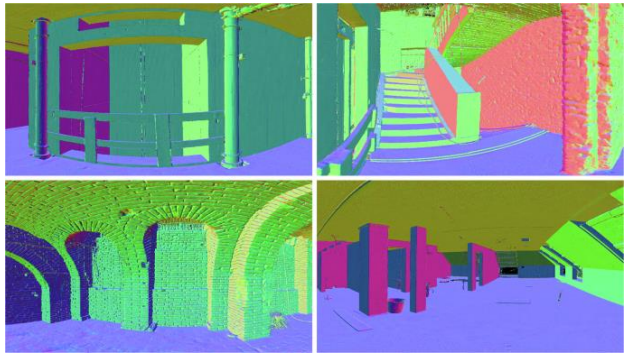
3D lidar single scan

Landmark Features for Lidar

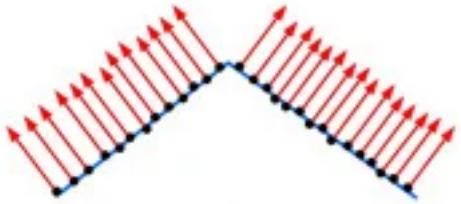
- **Detecting Corner Landmarks**

  1. Find edge points: Calculate normal vector for each point pair and group based on normal vector angle
     (may need a bit more sophistication -- e.g. angle histogram)
  2. Find observable corners, i.e. points where two edges intersect clearly
  3. Possibly refine corner position by fitting regression lines through edges and computing their intersection

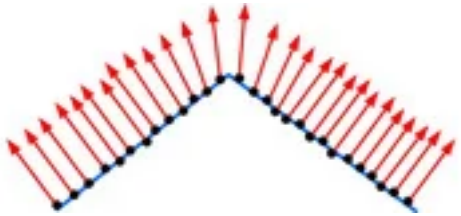
Of course this works in 3D too.



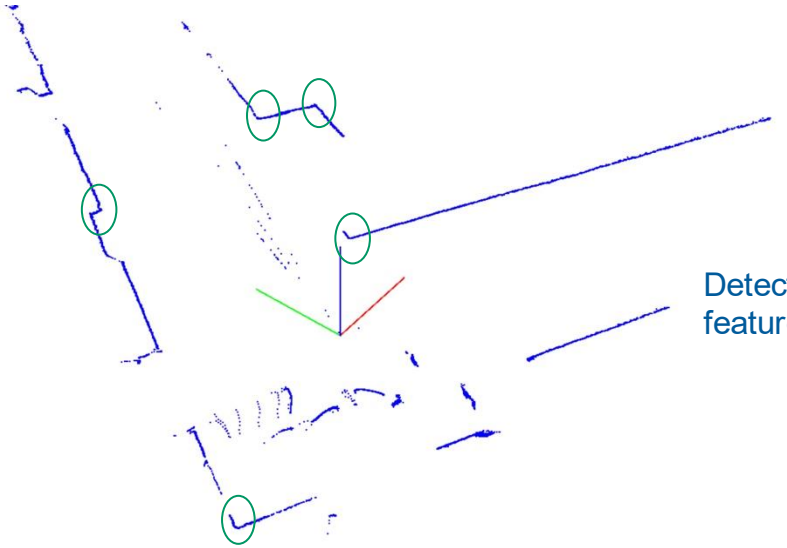
Surface normals for 3D lidar



Normal vectors for pairs of adjacent lidar points



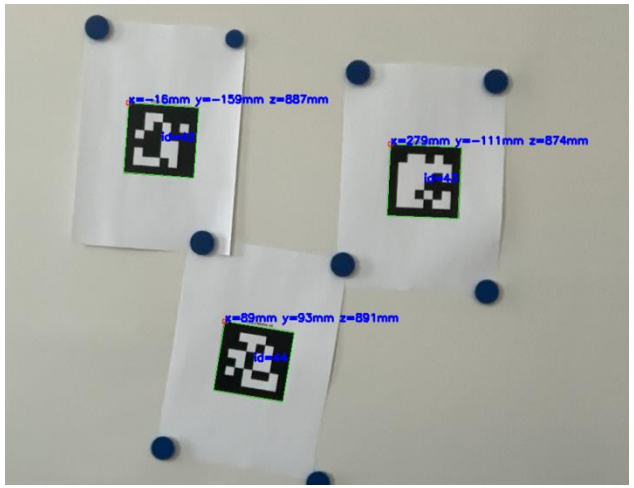
Possibly contain noise that makes direct segmentation difficult ➔ use histograms or Hough transform



Detected corner features

Landmark Features for Cameras: Artificial 3D reconstructable Landmarks

- **Cameras have the inherent problem of great visibility but no 3D information.**
- **Artificial landmarks like QR codes can overcome this problem:**

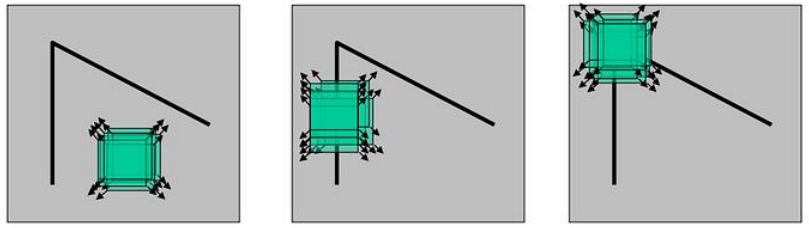


  - Easy to detect and identify, cheap
  - Each of the 4 corner gives an equation $(u, v, 1)^\top = K[R|t](x, y, z)^\top$, so 4*3=12 linear equations in total
  - Given camera calibration K, can be solved with least squares optimization for the 6 parameters in R and t

Landmark Features for Cameras: Recognizable Features

- **To use natural landmarks with cameras, we typically split the problems: We want landmarks that**
  1. can be easily recognized and associated to a map feature – create scale / transform invariant features.
  2. can be *localized to a single point* as precisely as possible – solve the 3D problem in some way.

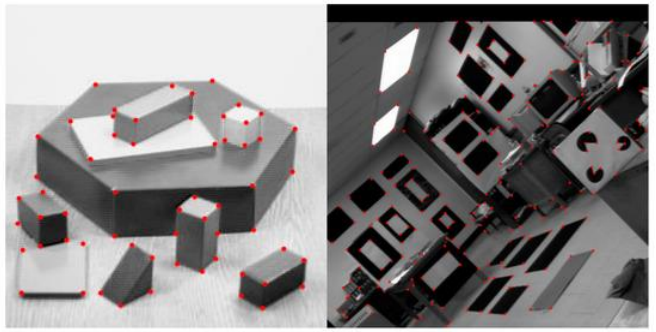
- **Let's look at creating features.**
- **The simplest way to detect 2D features is to look for "corners" – regions where the image gradient is high in multiple directions.**
- **See the section on image convolution with Sobel filters earlier in the lecture series.**
- **However these features are hard to match / store in a map, as they offer no feature description beyond pure 2D location.**



"flat" region: no change in all directions

"edge": no change along the edge direction

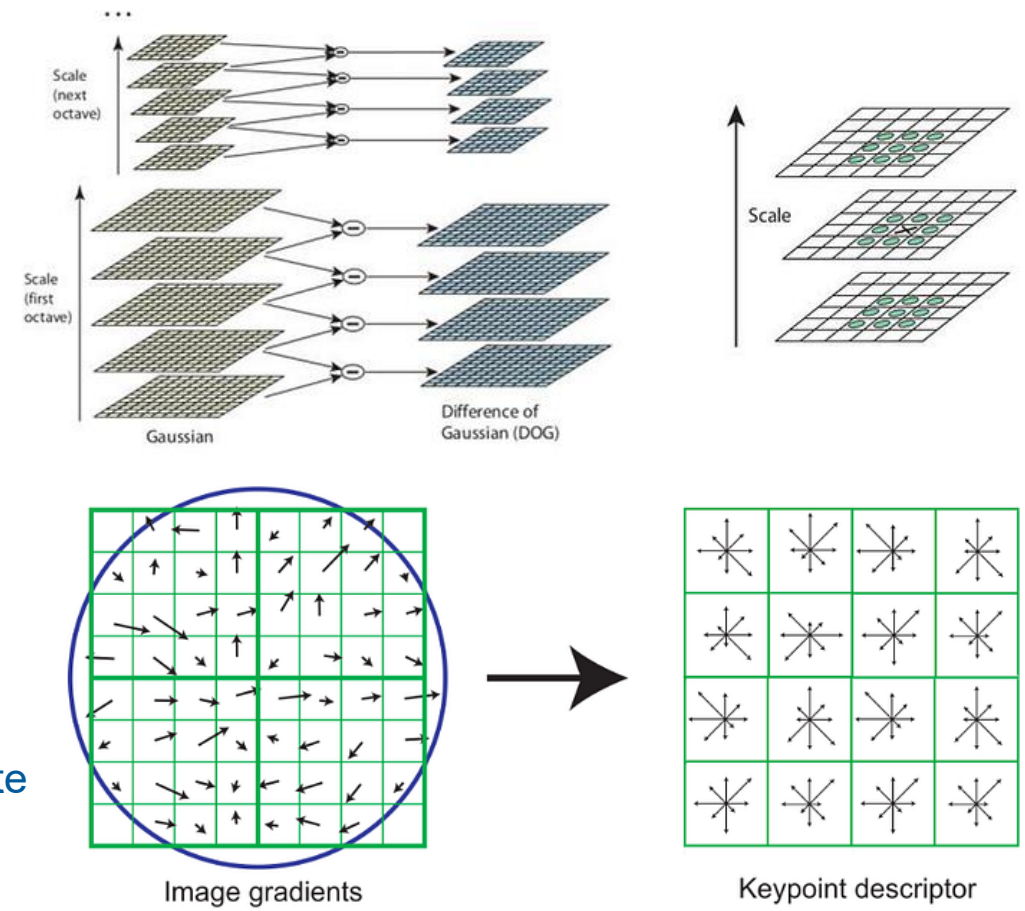
"corner": significant change in all directions



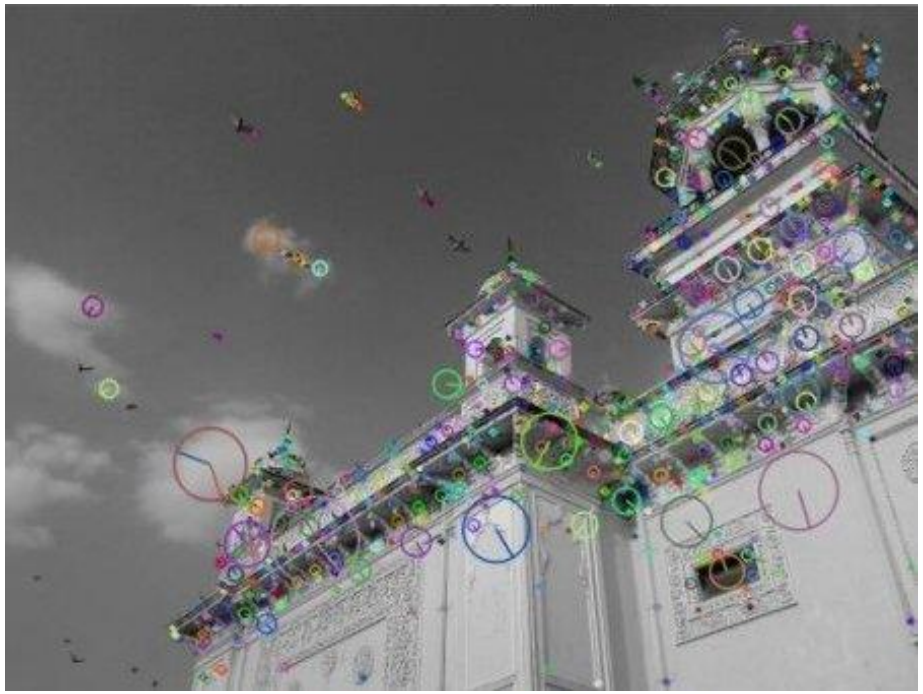
(a) (b)

Scale Invariant Feature Transform -- SIFT

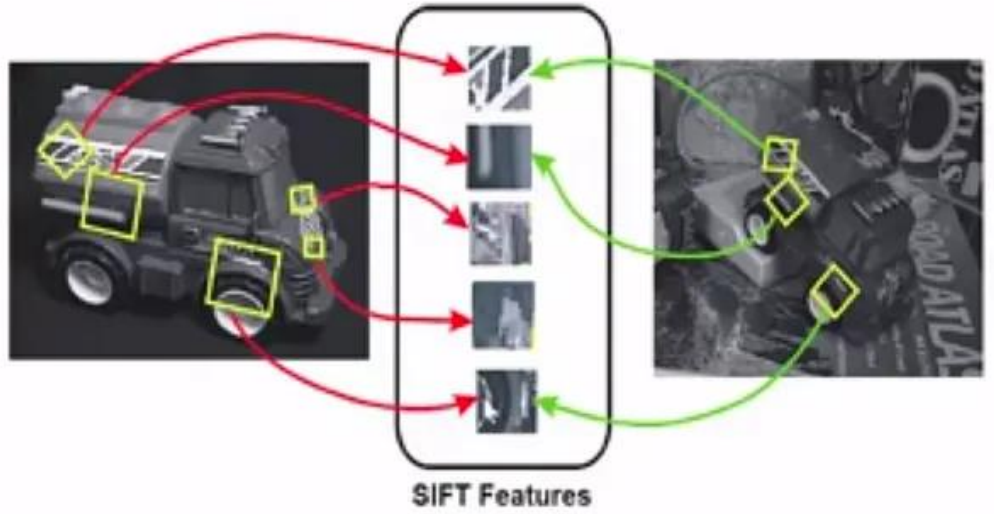
- **SIFT creates scale and 2D rotation invariant image keypoint descriptors.**
  - The original image is scaled down several times ("octaves")
  - On each scale level, the image is blurred with Gaussians of different radius. The resulting blurred images are subtracted from each other, giving difference images.
  - Intuition: Strong localized image features like corners will be more affected / smoothed by Gaussians than flat / single color structures.
  - Strong maxima / minima are then searched within a sliding window at each scale, yielding keypoints that have a sense of scale.
  - A 16x16 pixel region in the correct octave is then used to create a gradient histogram, yielding a keypoint descriptor.



Scale Invariant Feature Transform – SIFT



SIFT features



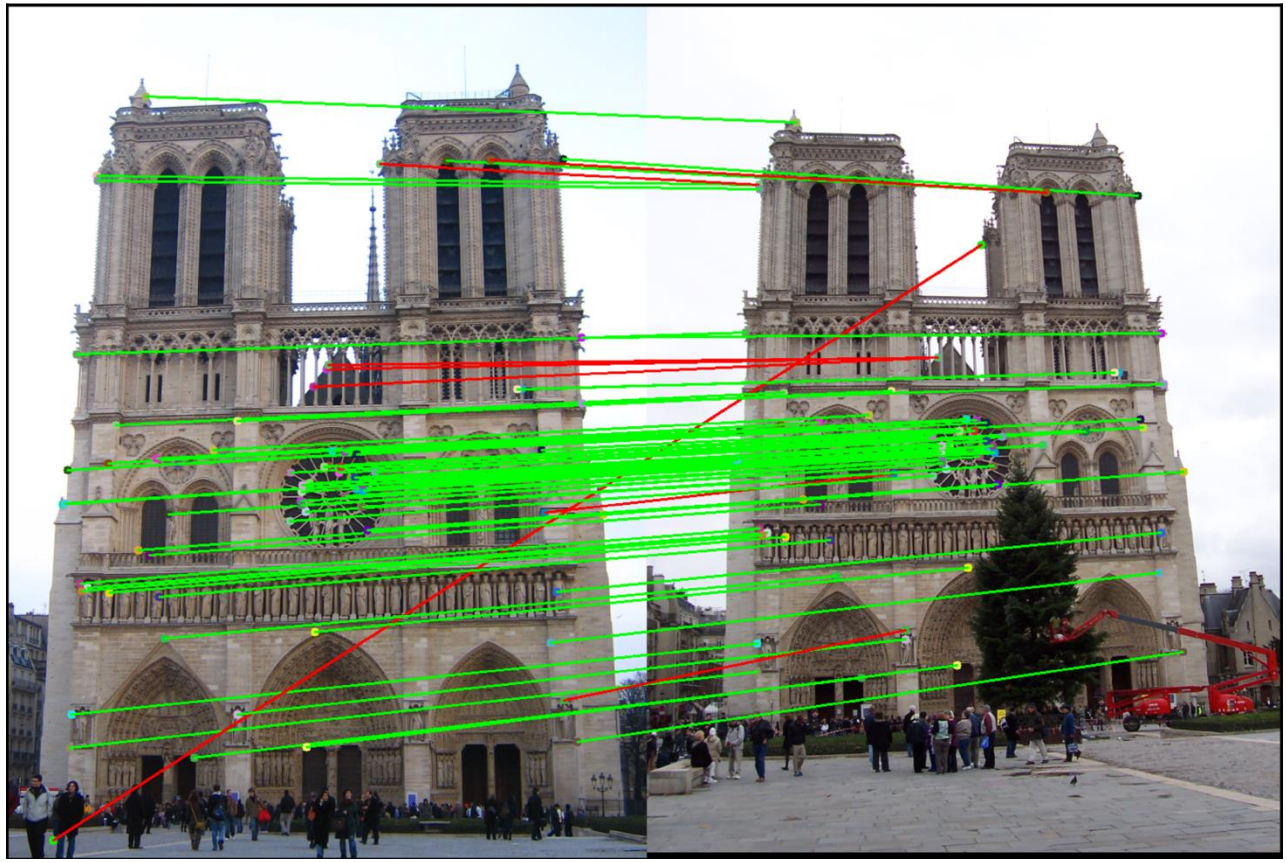
Detecting and matching SIFT features

- SIFT [3] was the first in a series of scale invariant keypoint detectors.
- It spawned a host of extensions and improvements, the most recent being ORB [4].

Keypoint-Based Feature Matching using SIFT / ORB

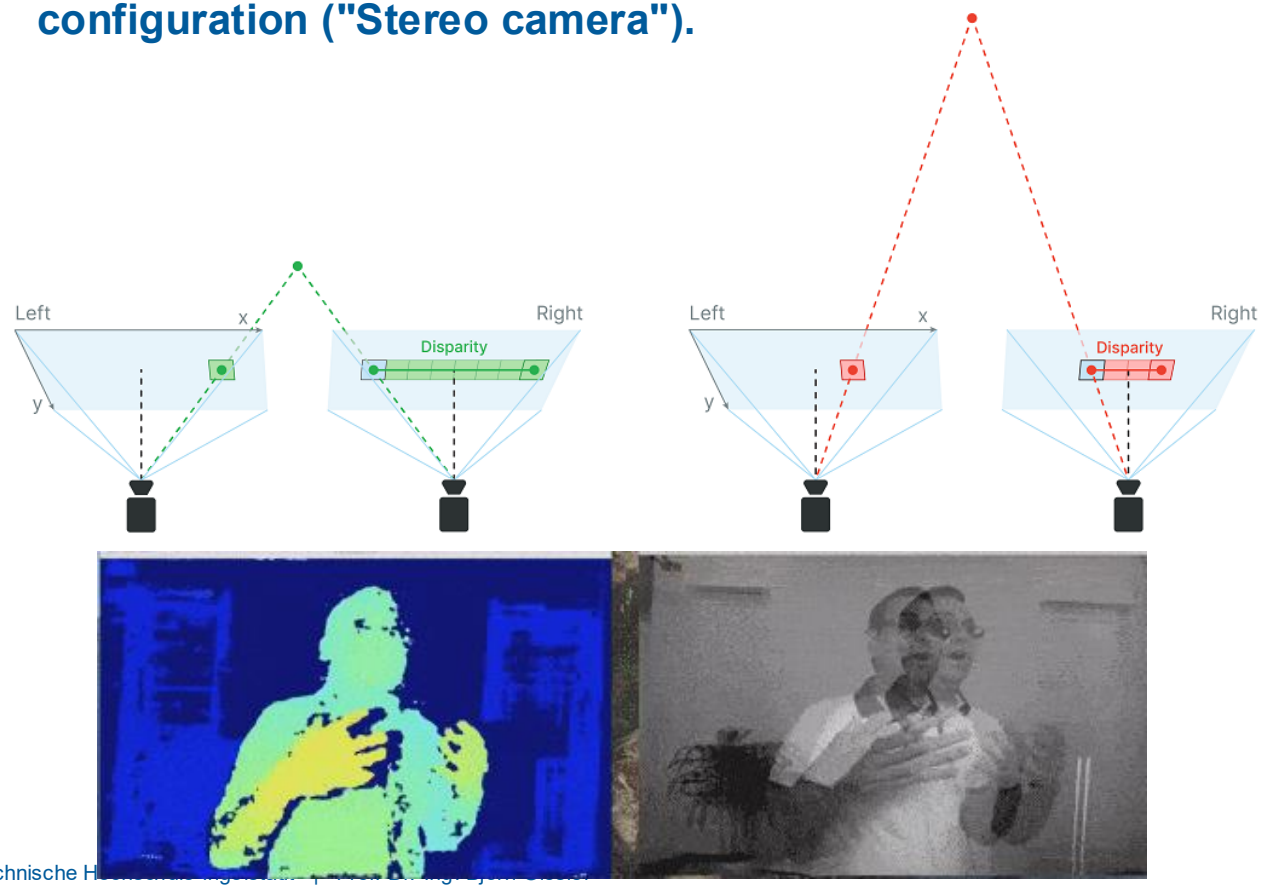
1. Extract features from image 1
2. Extract features from image 2
3. Find best matches
4. Apply your favorite sort of outlier rejection...

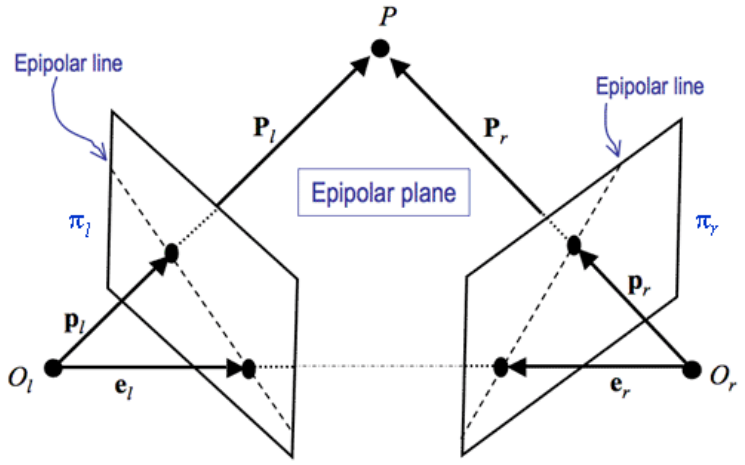
- 2D feature detectors still have the problem of being 2D – if something changes due to perspective shifts, they cannot recognize that.
- Individually error-prone
- Strong and computationally cheap enough to work well on a frame-by-frame basis with little motion.
- Still need 3D structure for SLAM...



Landmark Features for Cameras: 3D from Stereo

- **One way of solving the 3D-from-2D problem is to use a time-synchronized dual-camera system in a known configuration ("Stereo camera").**



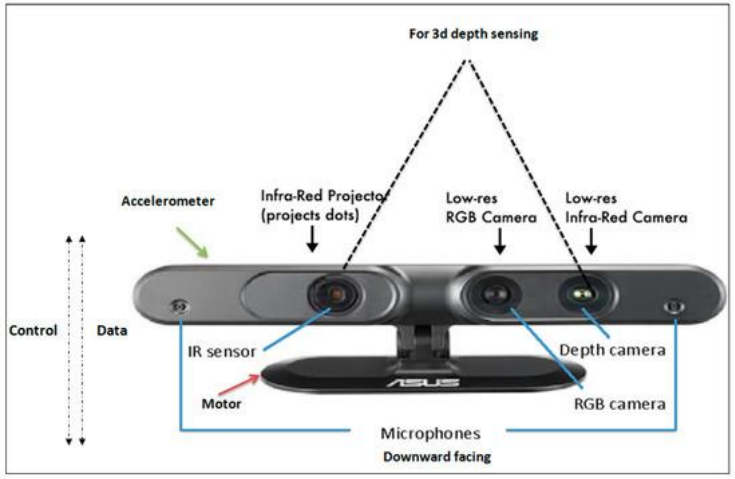


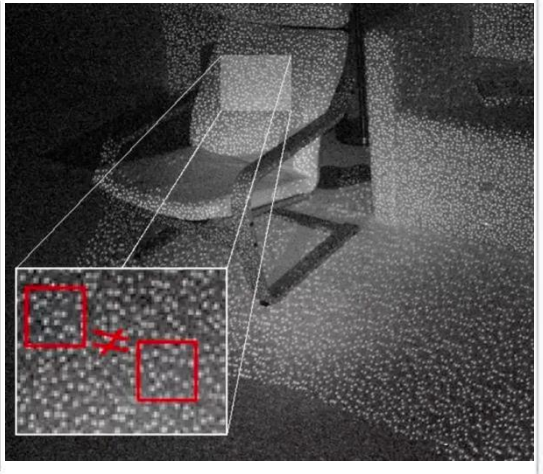
The *epipolar geometry* spanned by two cameras and a feature point gives a strong constraint for feature matching.

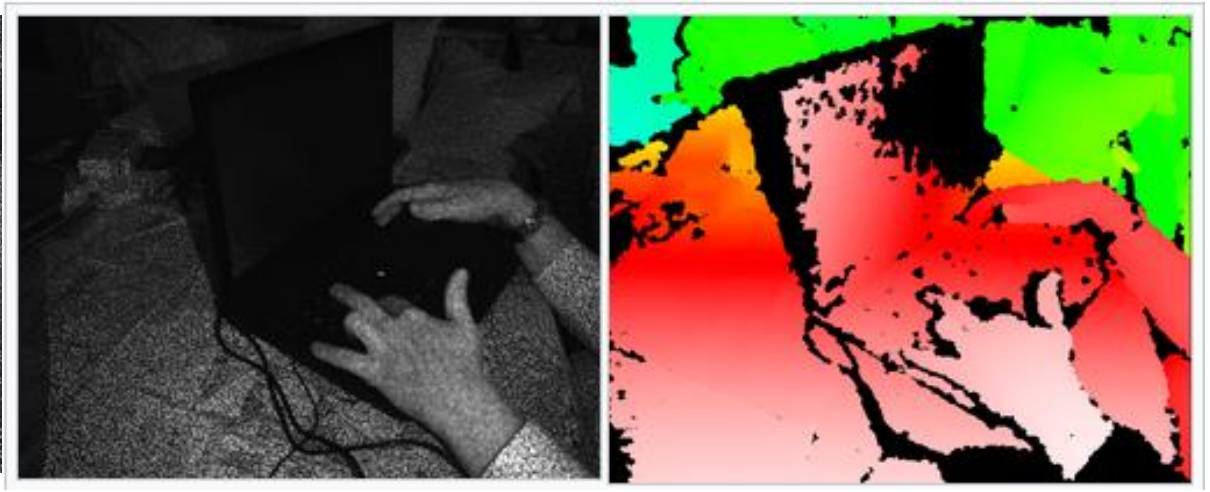
This can of course be extended to multi-view systems using more than 2 cameras.

Landmark Features for Cameras: RGBD Cameras

- Another way is using a RGBD camera, which combines a 2D video camera with a 3D sensor based on infrared pattern projection.
- A known stipple pattern is projected into the scene by an IR projector and recorded by an IR camera.
- The distortion between the known and recorded patterns give a measure of 3D structure.
- Together with the calibrated RGB camera, this yields an image with color and distance information.







Landmark Features for Cameras: Mono Camera SLAM

- **A HUGE amount of work has gone into trying to solve the problem of monoscopic camera SLAM.**
- **Most of it is in some way trying to compute a frame-by-frame estimate of the epipolar geometry of cameras at t and $t+1$.**
- **Some of it initializes its features from an initial frame.**
- **None of it works really well on its own.**
- **However the work led to significant contributions:**
  - using reprojection for more precise feature matching
  - tracking over multiple frames and then optimizing camera motion with multi-frame feature correspondences
  - integrating IMUs to optimize for fast motion
  - etc etc.

Andrew Davison: Real-Time 3D SLAM with Wide-Angle Vision, 2004



- Based on EKF-SLAM
- Tracking camera motion with an EKF, starting from 4 points on an initial known object
- Deriving depth of additional features with Particle Filters until "converged enough", then adding to the main EKF
- Aiding landmark recognition by projecting covariance ellipsoids into the image, then searching for best match in resulting projected ellipsis

ORB-SLAM3: Current State of the Art on camera-based SLAM



ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM

Carlos Campos*, Richard Elvira*, Juan J. Gómez Rodríguez,
José M. M. Montiel and Juan D. Tardós

Dataset: TUM VI Benchmark
Sequences: Room1+Magistrale1+Magistrale5+Slides1
Setup: Stereo-Inertial

Maps ORB scale-invariant features

Uses IMU motion in EKF if available

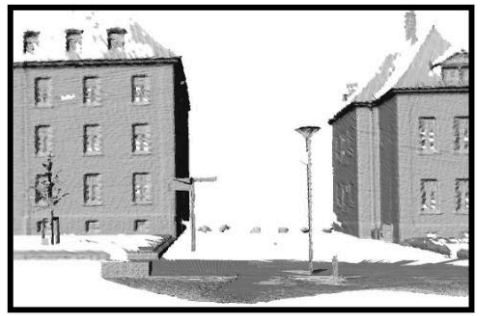
Uses stereo reconstruction if available

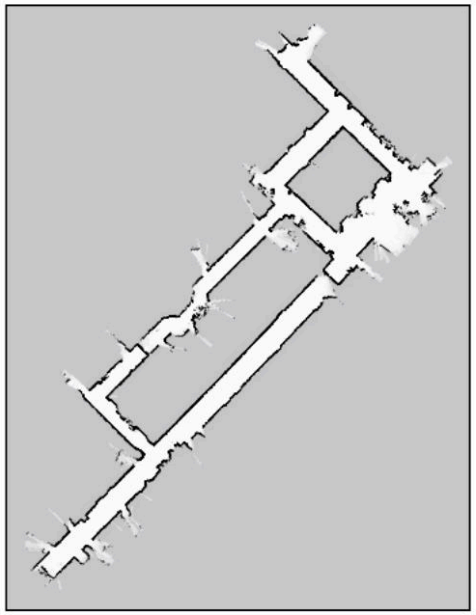
Uses RGBD depth if available

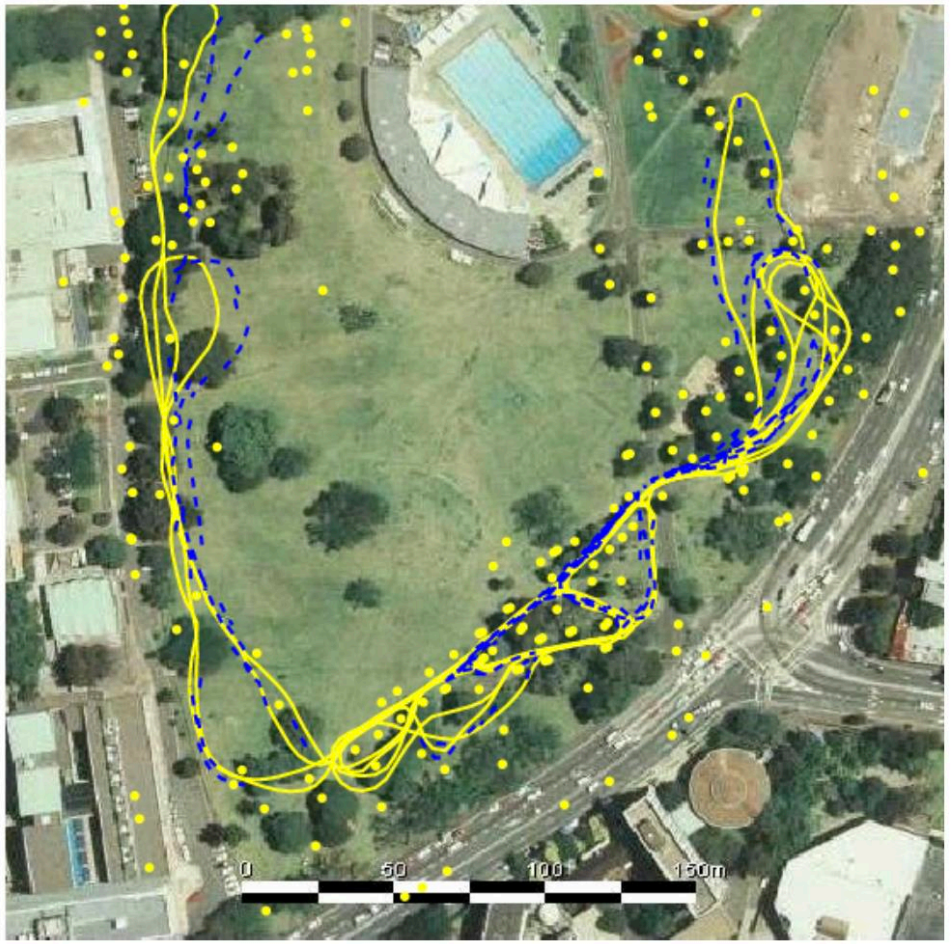
Recovers from tracking loss by starting new map, then incorporating if loop / match detected

Occupancy Grids and Grid-based SLAM

Feature Maps vs Volumetric Maps

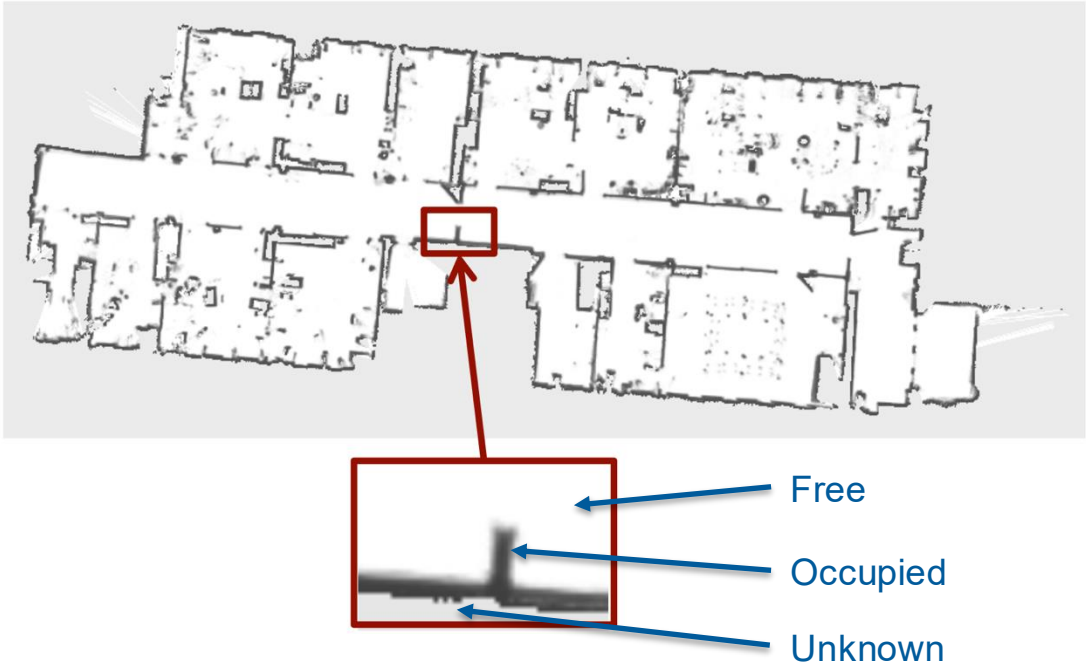






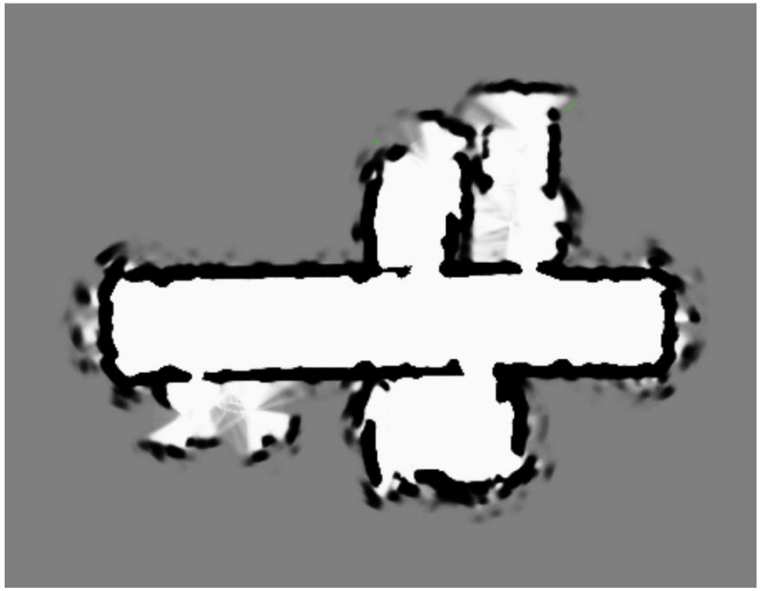
Features vs Grids

- **So far, we have only used feature maps.**
  - Natural choice for Kalman filter-based SLAM systems
  - Multiple feature observations improve the position estimate (EKF)
  - Compact representation
- **Grid maps:**
  - Discretize the world into cells
  - Grid structure is rigid
  - Each cell is assumed to be occupied or free space
  - Non-parametric model
  - Requires substantial memory resources
  - Does not rely on a feature detector



Occupancy Grid Mapping

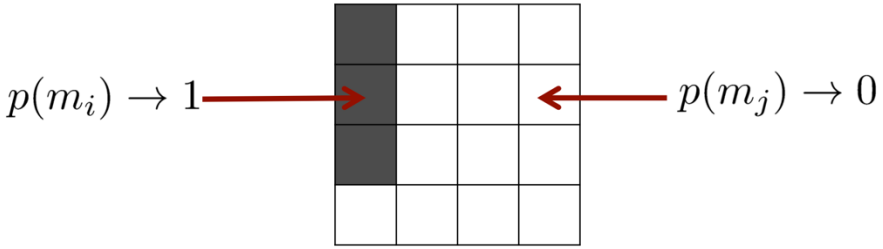
- Developed in the mid 80'ies by Moravec and Elfes
- Originally developed for noisy sonars
- Also called "mapping with known poses"





Assumptions

1. **The area that corresponds to a cell is either completely free or occupied**
   - We can model each cell with a binary random variable representing the occupancy
   - Cell is occupied: $p(m_i) = 1$, not occupied: $p(m_i) = 0$, no knowledge: $p(m_i) = 0.5$
   - The state is assumed to be static
2. **The cells (= random variables) are independent of each other**
3. **We have a sensor model that can measure free and occupied space**
4. **The likelihood $l_{t,i}$ of a cell being free can be represented by**



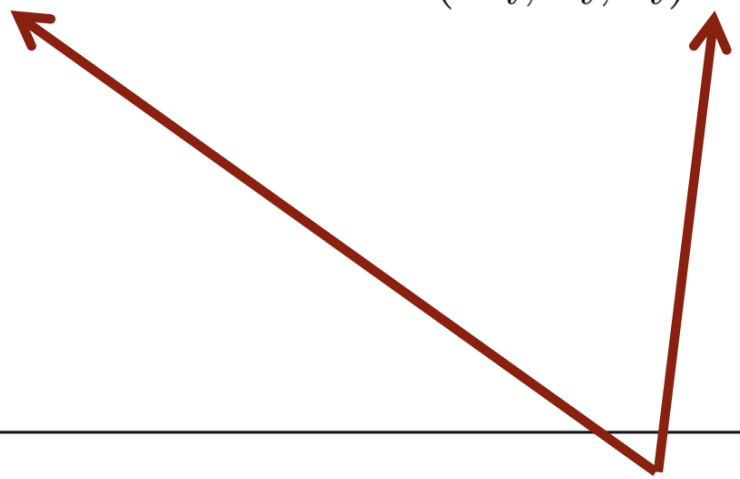
$$l(m_i|z_{1:t}, x_{1:t}) = \underbrace{l(m_i|z_t, x_t)}_{\text{Inverse sensor model}} + \underbrace{l(m_i|z_{1:t-1}, x_{1:t-1})}_{\text{recursive term}} - \underbrace{l(m_i)}_{\text{prior}}$$

or in short $l_{t,i} = \text{inv_sensor_model}(m_i, x_t, z_t) + l_{t-1,i} - l_0$

Occupancy Mapping Algorithm

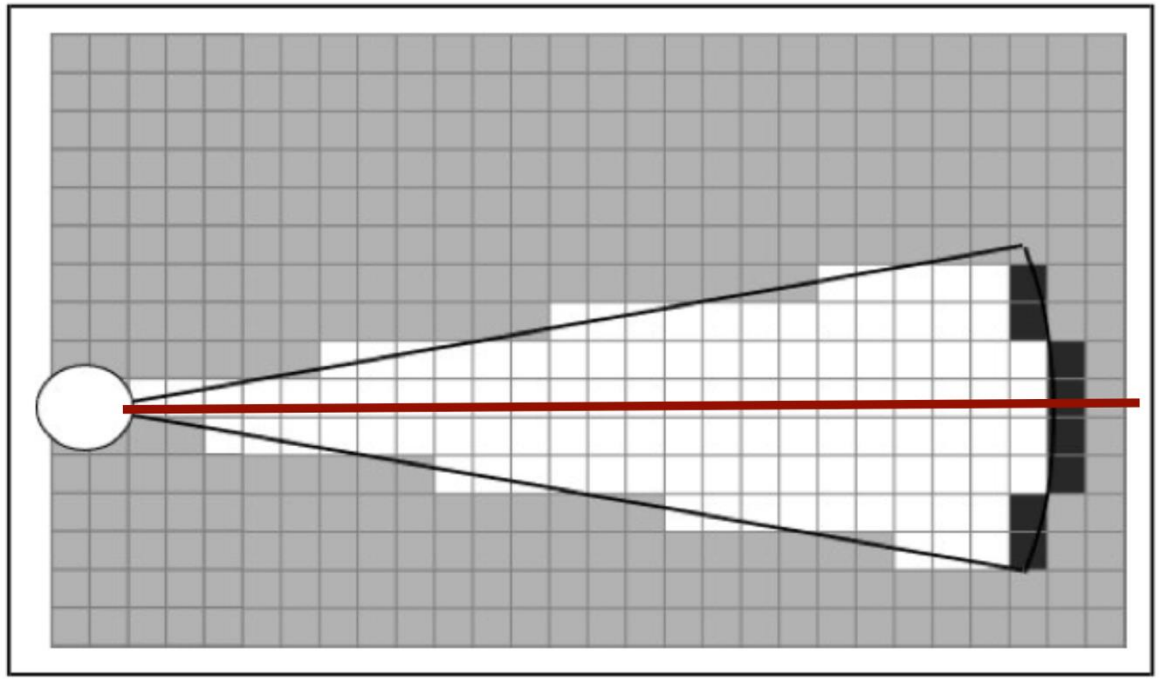
occupancy_grid_mapping$(\{l_{t-1,i}\}, x_t, z_t)$**:**

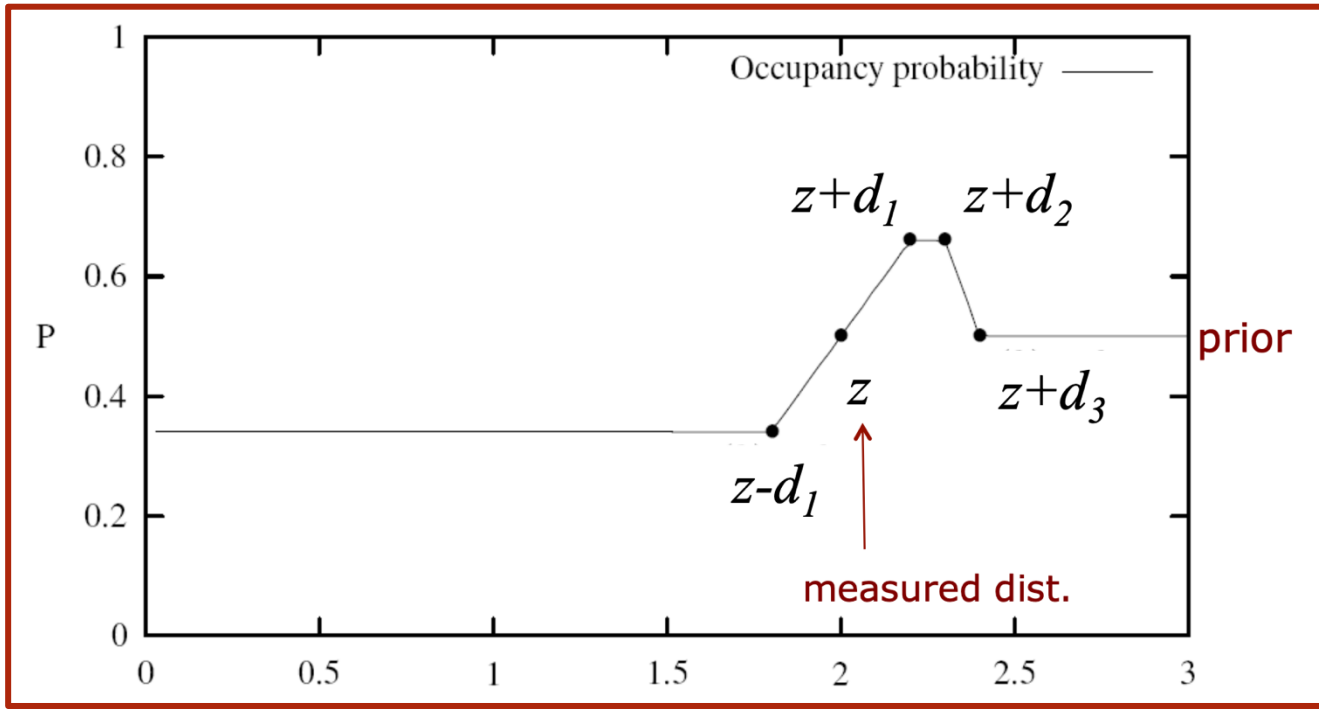
1: *for all cells* m_i *do*
2: *if* m_i *in perceptual field of* z_t *then*
3: $l_{t,i} = l_{t-1,i} + \text{inv_sensor_model}(m_i, x_t, z_t) - l_0$
4: *else*
5: $l_{t,i} = l_{t-1,i}$
6: *endif*
7: *endfor*
8: *return* $\{l_{t,i}\}$



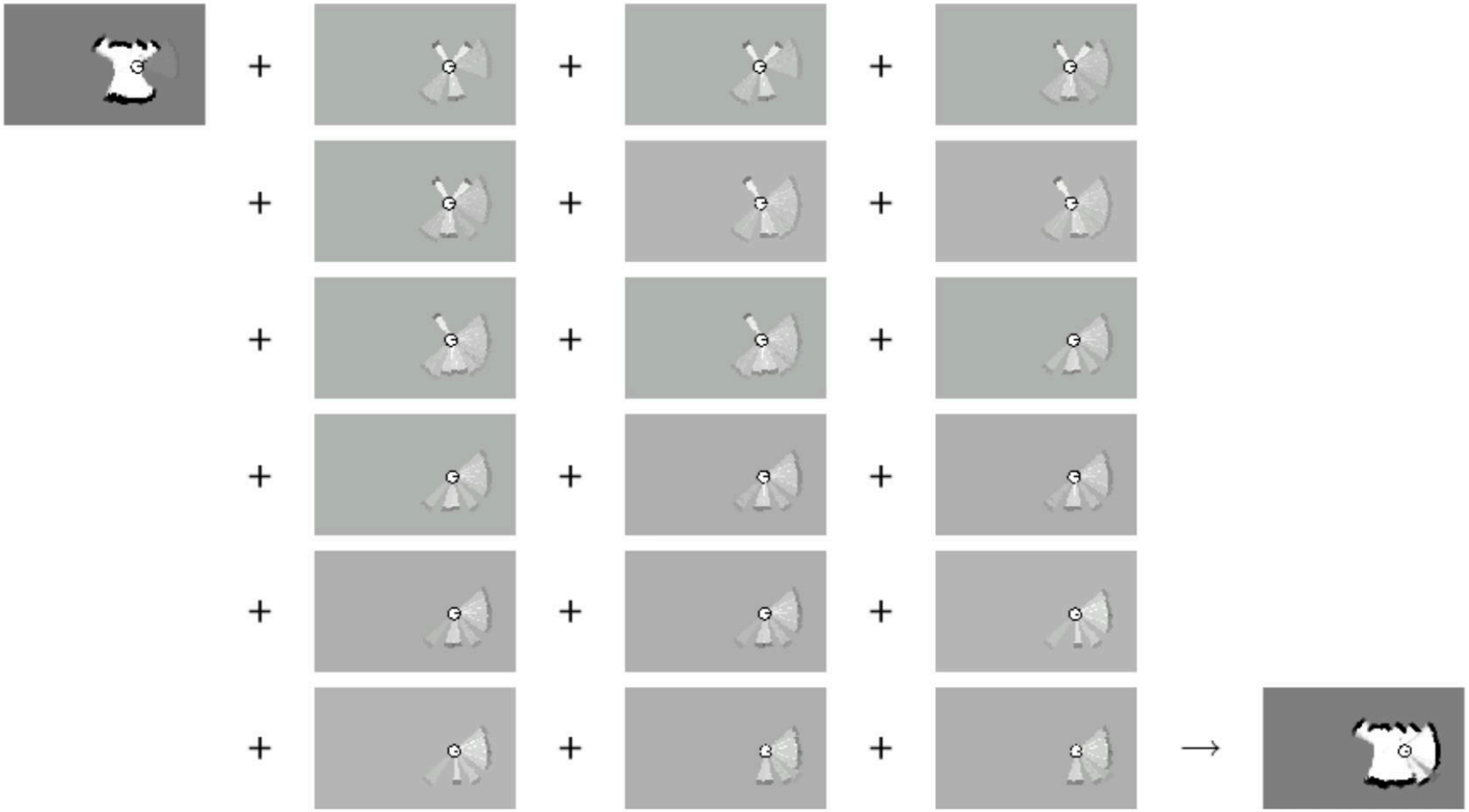
highly efficient, only requires to compute sums

Inverse Sensor Model for Sonar Range Sensors

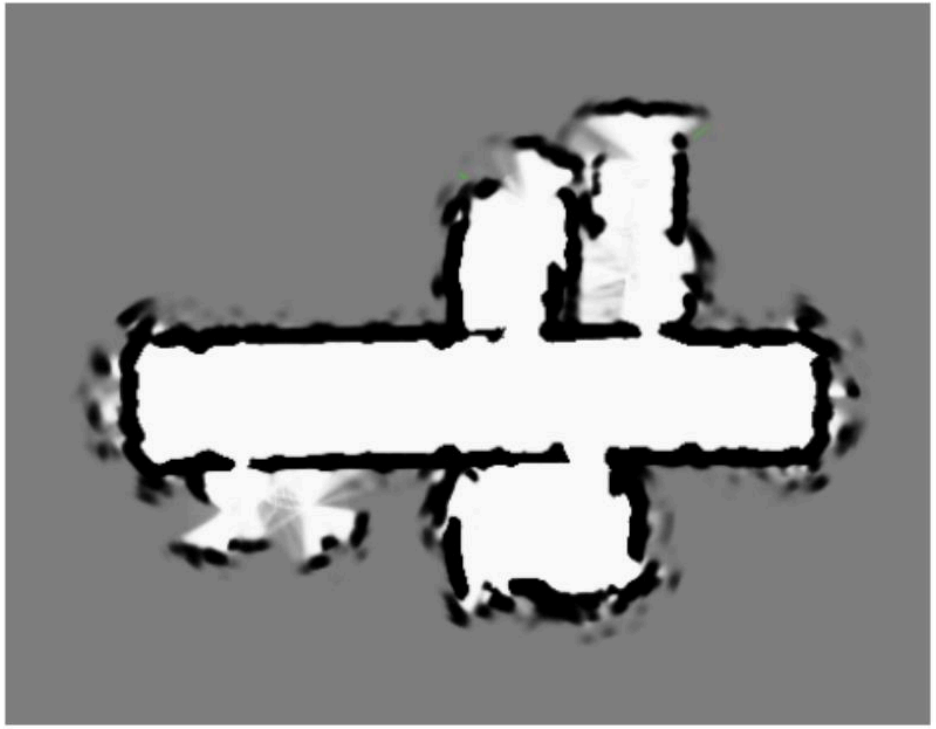


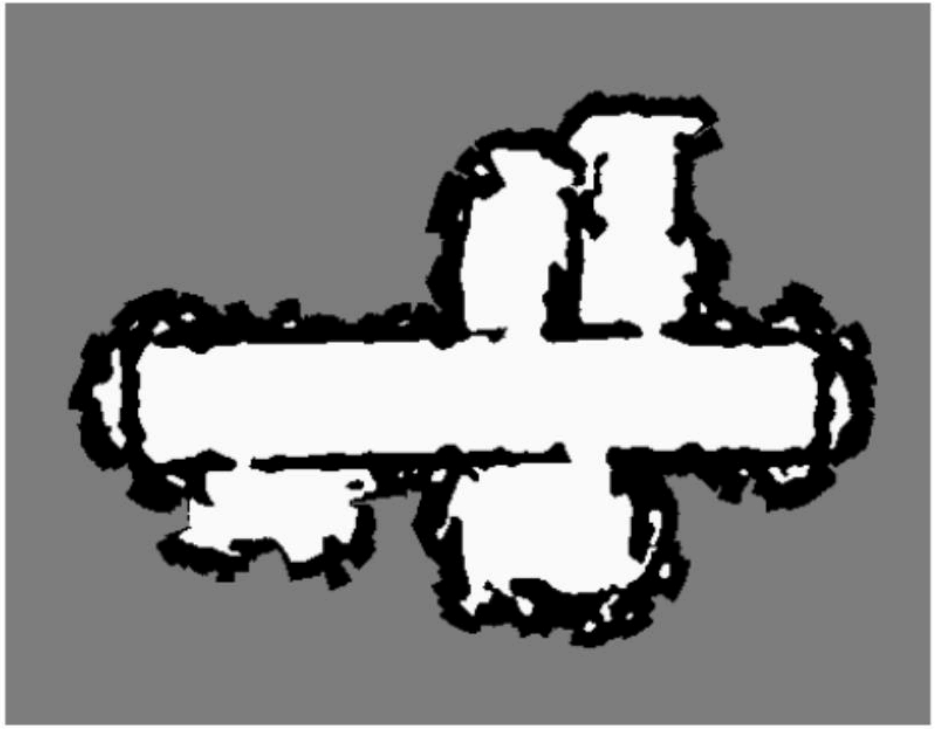


Example: Incremental Update on Occupancy Grids



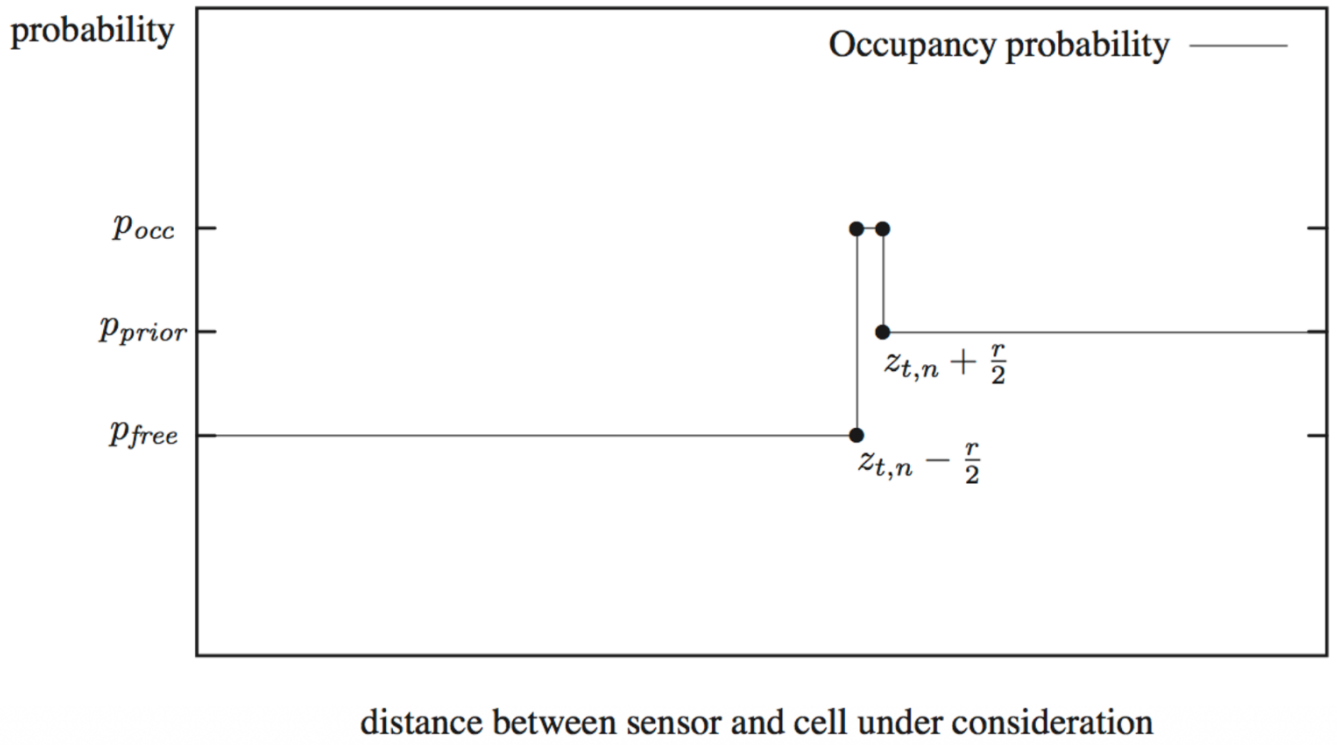
Resulting Occupancy Grid and Maximum Likelihood Map



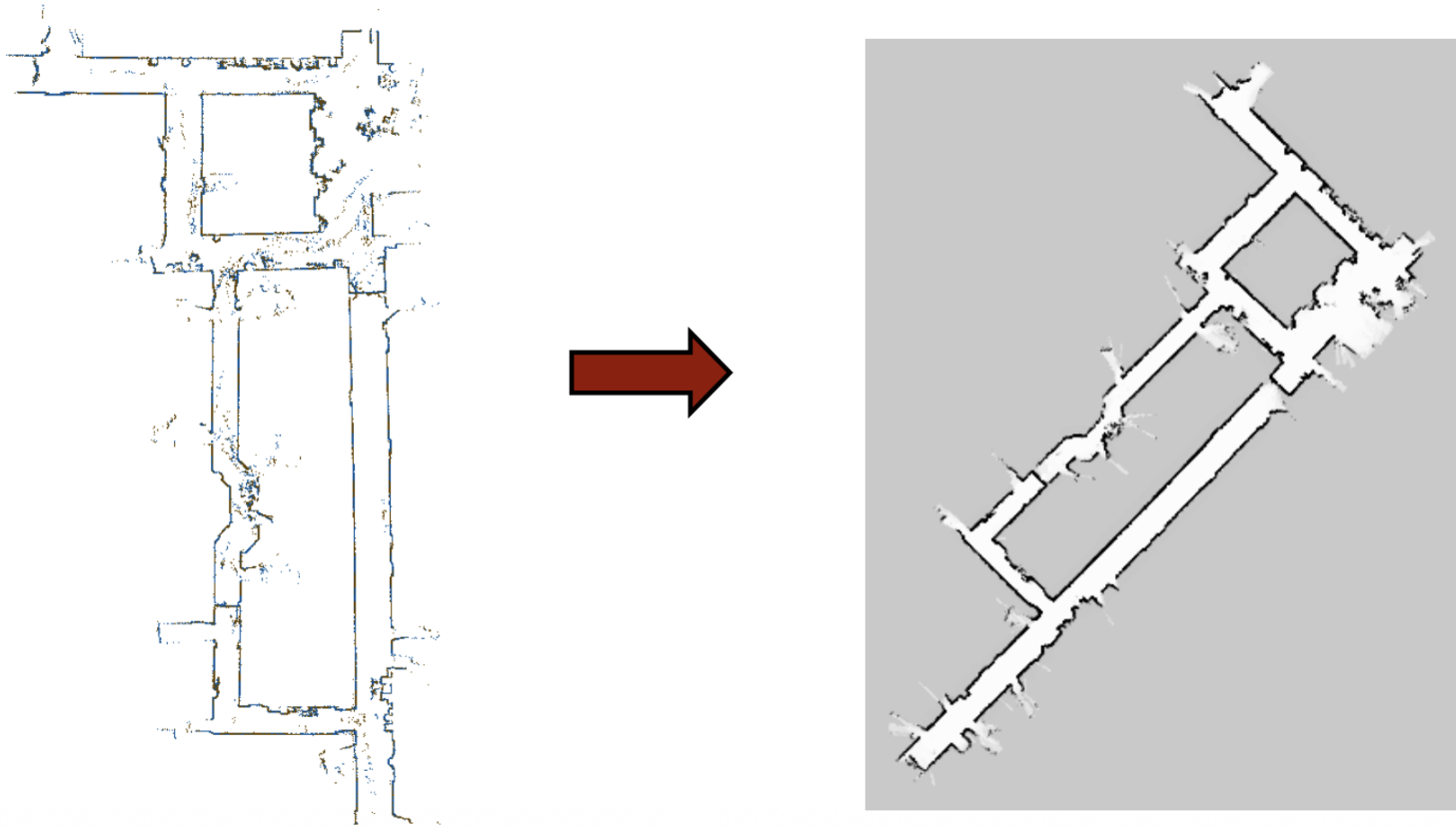


- **The maximum likelihood map is obtained by rounding the probability for each cell to 0 or 1.**

Inverse Sensor Model for Laser Range Finders



From Laser Scans to Maps



Summary

- Occupancy grid maps discretize space into independent cells
- Each cell is a binary random variable estimating if the cell is occupied
- Static state binary Bayes filter per cell
- Mapping with known poses is easy, cell states are fast to compute
- No need for predefined features
- However, only solves the mapping part ("known poses"), not the localization part.

- **Previous lectures:**
  - Mathematical details of Particle Filters
  - Joint Probability Data Association Filters
  - JPDAs and Particle Filters
  - Simultaneous Localization and Mapping (based on Kalman Filters)
- **This week:**
  - Simultaneous Localization and Mapping (based on Particle Filters)
  - Overview over features for SLAM
  - Occupancy Grids and Grid-based SLAM
- **Upcoming:**
  - (DNN-based) early fusion approaches.

Literature / Links

1. A. Doucet &al., ***Rao-Blackwellised Particle Filtering for Dynamic Bayesian Networks***, https://arxiv.org/pdf/1301.3853
2. M. Montemerlo &al., ***FastSLAM: A Factored Solution to the Simultaneous Localization and Mapping Problem***, https://robots.stanford.edu/papers/montemerlo.fastslam-tr.pdf
3. D. Lowe: ***Distinctive Image Features from Scale-Invariant Keypoints,*** https://link.springer.com/article/10.1023/B:VISI.0000029664.99615.94
4. E. Rublee & al.: ***ORB: an efficient alternative to SIFT or SURF,*** https://www.researchgate.net/publication/221111151_ORB_an_efficient_alternative_to_SIFT_or_SURF



Technische Hochschule Ingolstadt

Fakultät Elektro- und Informationstechnik

Thank you very much!